

# Core Concepts — LangGraph

Track 16.4 —  LangGraph · 16-ai-tech-stack/tracks/langgraph/notes/01-core-concepts.md

Verified against **LangGraph 1.2.x** (`pip show langgraph`), September 2026. Primary source for everything below: <https://docs.langchain.com/oss/python/langgraph/graph-api>

## 1. The thesis: an agent is a graph with state, not a chain

A chain is a DAG of prompts. Data goes in one end, comes out the other, and every edge was decided before the run started. That model is fine for "summarise this document" and falls apart the moment you need any of these five things:

1. **Loops** — try, check the result, try again. A DAG cannot express "go back".
2. **Branching on model output** — the model decides what happens next, at runtime.
3. **Retries with memory of the failure** — the second attempt should know why the first failed.
4. **Pausing** — a human has to approve this before it touches money.
5. **Resuming** — the process died / the deploy rolled / the user came back tomorrow.

People discover this in the same order every time. First they write a chain. Then they wrap it in a `while` loop. Then they add a step counter. Then a `try/except` that re-prompts. Then a dict of "stuff so far" threaded through every function. Then someone asks for an approval step and they discover their whole loop lives in one process's stack, which means approval means *blocking a worker for three hours*. At that point you have written a bad workflow engine.

LangGraph is that workflow engine, written properly. Its bet: **model your agent as a state machine whose transitions are explicit and whose state is persisted after every step**. Nodes do work, edges decide control flow, and a checkpointer writes the whole state to durable storage between steps.

|                                                                                                      |                                                                                                                                                                                                                                      |
|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>chain (DAG)</p> <p>A → B → C → out</p> <p>fixed at authoring time</p> <p>one process, one run</p> | <p>graph (LangGraph)</p> <p>START → agent → [route?] → tools →+</p> <p style="text-align: center;">^</p> <p style="text-align: center;">+-----+</p> <p>route? → END   tools   human_gate</p> <p>state persisted after EVERY step</p> |
|------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Underneath it is **Pregel** — Google's bulk-synchronous parallel graph model. Execution proceeds in **supersteps**: every node scheduled for this step runs (in parallel), all their writes are merged into state, then the next step is scheduled. That is why parallel branches "join" automatically, why `recursion_limit` counts supersteps rather than node calls, and why two nodes writing the same channel in the same superstep is an error rather than a race.

**Where you are:** `../../06-ai-agents/` taught you the ReAct loop and tool use; `../../07-agentic-ai/` taught you multi-agent topologies. This track is the *runtime* those ideas need to survive contact with production. `../langchain/` is the model/tool abstraction layer LangGraph sits on top of, and `../langsmith/` is how you see what the graph actually did.

## 2. State: the part that is actually hard

Everything else in LangGraph is an hour of reading. **State design is the work.**

A graph is parameterised by a state schema — a `TypedDict`, a dataclass, or a Pydantic model:

```
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph.message import add_messages
import operator

class State(TypedDict):
    messages: Annotated[list, add_messages] # appended, id-aware
    findings: Annotated[list[str], operator.add] # appended
    plan: str # overwritten
    attempts: int # overwritten
```

Each key is a **channel**. Nodes return a *partial* dict and LangGraph merges it channel by channel using that channel's **reducer**:

| Declaration                                | Channel type           | Merge behaviour                              |
|--------------------------------------------|------------------------|----------------------------------------------|
| <code>x: str</code>                        | <code>LastValue</code> | the update <b>replaces</b> the old value     |
| <code>Annotated[list, operator.add]</code> | binary op              | <code>old + update</code> — append           |
| <code>Annotated[list, add_messages]</code> | custom                 | append, but <b>replace by message id</b>     |
| <code>Annotated[T, my_fn]</code>           | custom                 | <code>my_fn(old, update)</code> — you decide |

Three rules that will save you a week:

- **A missing reducer is a silent overwrite.** If two nodes both write `findings` and you forgot `Annotated`, the second one wins and nobody tells you. Under *parallel* branches you at least get a loud `InvalidUpdateError: Can receive only one value per step` — run `code/graph_basics.py` section 4 to watch it happen.
- **`add_messages` is not `operator.add`.** It matches on message `id`, which is what lets you edit, redact, or patch history instead of only appending. This is the mechanism behind "remove this tool call from the transcript before the next model call".
- **Ownership beats cleverness.** The cleanest fix for a merge conflict is usually not a fancier reducer — it is deciding that exactly one node owns that channel.

## Pydantic vs. TypedDict

`TypedDict` is the default and is *not* validated at runtime — it is a type-checker hint. Use a Pydantic `BaseModel` as the state schema when you want the graph to reject a malformed node return at the boundary; pay for it in serialisation overhead on every checkpoint write. Practical advice: `TypedDict` for internal state, Pydantic for the `input_schema/output_schema` you expose to callers (`StateGraph(State, input_schema=Request, output_schema=Answer)`).

## What does *not* belong in state

State is checkpointed after every superstep. Everything you put there is serialised, stored, and replayed. So keep out: DB connections, HTTP clients, secrets, 40 MB of retrieved documents, and anything you would be unhappy to find in an audit export.

Where to put it instead: - **Static per-run config** (user id, tenant, model name) → the **context** (`StateGraph(State, context_schema=Ctx)`, read via `Runtime.context` in a node). In LangGraph v1 this replaced stuffing things into `config["configurable"]`. - **Cross-thread memory** (user preferences, learned facts) → a **Store** (from `langgraph.store.memory import InMemoryStore`), which is namespaced and *not* thread-scoped. - **Big blobs** → object storage; put the URI in state.

## 3. Topology: nodes, edges, and the four ways to route

```
from langgraph.graph import StateGraph, START, END

b = StateGraph(State)
b.add_node("plan", plan_fn)          # a node is (state) -> dict. That's it.
b.add_edge(START, "plan")           # unconditional
b.add_conditional_edges("plan", route, ["research", "answer"]) # dynamic
b.add_edge("answer", END)
graph = b.compile()
```

| Mechanism                                           | Use it when                                       | Note                                                                                                |
|-----------------------------------------------------|---------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>add_edge(a, b)</code>                         | control flow is fixed                             | two edges out of one node = <b>parallel</b> , same superstep                                        |
| <code>add_conditional_edges(a, fn, path_map)</code> | the routing decision is data                      | <code>fn</code> returns a node name, <code>END</code> , or a list of names                          |
| <code>Command(update={...}, goto="x")</code>        | the node that did the work also knows where to go | returned <i>from</i> the node; keeps routing logic next to the reason for it                        |
| <code>Send("worker", {...})</code>                  | fan-out width is only known at runtime            | map-reduce; each <code>Send</code> gets its <b>own</b> state payload and may use a different schema |

Two details people miss:

- The `path_map` third argument is optional for execution but not for you. Without it, `graph.get_graph()` and LangGraph Studio cannot draw the possible destinations, so your cycle renders as

a mystery. Always pass it (or use `destinations=` on `add_node` when routing with `Command`).

- `Command(goto=..., graph=Command.PARENT)` is how a node inside a subgraph hands control back to a sibling in the parent graph. That one line is the entire "hand-off" primitive that multi-agent frameworks charge you a whole abstraction for.

## Cycles and how they end

A cycle is just an edge pointing backwards. The interesting question is what stops it. There are exactly three stoppers, and you want the first two:

1. **A semantic exit** — the model returned no tool calls; the judge scored  $\geq$  threshold.
2. **A budget in state** — `if state["attempts"] >= MAX: return END`. Inspectable, loggable, explainable to a user ("stopped after 6 tool calls").
3. **recursion\_limit** (default 25 supersteps) — raises `GraphRecursionError`. This is a circuit breaker, not a design. If it fires in production you already paid for 25 model calls.

## Subgraphs

A compiled graph is a valid node: `parent.add_node("researcher", research_graph)`. Two cases:

- **Shared state keys** — pass the compiled graph directly; the parent's channels flow in and out.
- **Different schemas** — wrap it in a function that translates: `lambda s: {"summary": sub.invoke({"query": s["question"]})["result"]}`. Do this more often than you think; it is how you keep an agent's noisy internal `messages` out of the parent's state.

Subgraphs get their own checkpoint namespace, which is why you need `subgraphs=True` on `.stream()` to see inside them and why `get_state(..., subgraphs=True)` exists.

---

## 4. The comparison you actually need

| Option                                | Gives you                                                                        | Costs you                                                                         | Choose it when                                                                        |
|---------------------------------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| A plain while loop                    | zero deps, total clarity, trivial debugging                                      | no persistence, no resume, no HITL, no parallelism; you rebuild all of it by hand | single-turn tools, scripts, prototypes, or a loop with <5 steps and no approval needs |
| create_agent (LangChain 1.x prebuilt) | the ReAct loop + middleware, in one call; still a LangGraph graph underneath     | opinionated topology; customisation goes through middleware hooks                 | the tool-calling loop is the product                                                  |
| LangGraph StateGraph                  | arbitrary topology, durable state, interrupts, time travel, streaming, subgraphs | a real learning curve; state design is on you; verbose for simple things          | loops + approval + resume + multi-step state — i.e. anything a business depends on    |
| Temporal / Restate / Airflow          | industrial durable execution, mature ops tooling                                 | not LLM-aware; no message state, no streaming tokens, no agent primitives         | the LLM is one step in a long business workflow, not the workflow                     |
| CrewAI / AutoGen / OpenAI Agents SDK  | fast to a demo; role-based or conversation-based abstractions                    | less control over the loop; persistence and HITL are shallower                    | prototypes, research, or when the built-in pattern is exactly your problem            |

**My default:** start with a plain loop. Move to `create_agent` when you want tool-calling plumbing you did not write. Move to `StateGraph` the first time someone says "can a human check this before it sends" or "what happened on run #4471" — because those two questions are what the framework is *for*, and retrofitting them is much worse than starting with them.

**When NOT to use LangGraph:** a one-shot classification, a RAG pipeline that is genuinely linear, or any system where you would struggle to name the state. If your state schema is `{"input": str, "output": str}`, you have written a function with extra steps.

## 5. A worked mental model: the ReAct loop as a graph

```
START --> agent --(tool_calls? no )-----> END
      ^                                     ^
      |                                     | budget spent
      +--(yes)--> [approval_gate] --> tools -----+
                  ^ interrupt()
```

That is `code/react_agent_from_scratch.py` [16-ai-tech-stack/tracks/langgraph/code/react\_agent\_from\_scratch.py] plus one node. Everything a production agent needs hangs off those five boxes:

- **agent** — one model call. Swap the model per node if you want a cheap router and an expensive executor.
- **the router** — three exits, not one: model finished, budget spent, or keep going.

- **tools** — executes every tool call on the last message and returns one `ToolMessage` per `tool_call_id`, always. Drop one and the next provider call fails schema validation.
- **error handling** — a failing tool returns `ToolMessage(status="error")` so the model can fix its own arguments. Only let a node *raise* for things a retry could fix.
- **approval\_gate** — an `interrupt()` between decision and action. Four lines; see `notes/02-production-craft.md` [16-ai-tech-stack/tracks/langgraph/notes/02-production-craft.md].

## 6. The vocabulary, tightened

| Term                           | Precise meaning                                                                     |
|--------------------------------|-------------------------------------------------------------------------------------|
| <b>channel</b>                 | one key of the state schema, with its own reducer and version counter               |
| <b>reducer</b>                 | <code>(old, update) -&gt; new</code> for a channel; absent means overwrite          |
| <b>superstep</b>               | one Pregel round: schedule, run in parallel, merge writes, repeat                   |
| <b>node</b>                    | <code>(state) -&gt; partial state</code> ; optionally <code>(state, runtime)</code> |
| <b>edge / conditional edge</b> | static successor / a function returning successor name(s)                           |
| <b>Send</b>                    | a dynamic task: node name + its own input state                                     |
| <b>Command</b>                 | a node return that combines a state update with a <code>goto</code>                 |
| <b>checkpointer</b>            | writes a state snapshot per superstep, keyed by thread                              |
| <b>thread</b>                  | one conversation/run lineage, addressed by <code>thread_id</code>                   |
| <b>checkpoint</b>              | one immutable snapshot inside a thread, addressed by <code>checkpoint_id</code>     |
| <b>interrupt</b>               | a resumable pause raised from inside a node                                         |
| <b>store</b>                   | cross-thread key-value memory, independent of checkpoints                           |

## 7. Where this returns

| Idea here                                  | Where it comes back                                                          |
|--------------------------------------------|------------------------------------------------------------------------------|
| ReAct loop, tool schemas                   | <code>../06-ai-agents/</code> — the pattern this graph implements            |
| Supervisor / hand-offs / hierarchies       | <code>../07-agentic-ai/</code> — the topology catalogue                      |
| Approval gates, audit trails, retention    | <code>../12-ai-governance/</code> — why checkpoints are evidence             |
| Tool-call injection, over-permissive tools | <code>../13-ai-security/</code> — the gate is also a control                 |
| Judging a node's output, regression suites | <code>../15-ai-evals/</code> — the <code>critique</code> node, done properly |
| Retrieval nodes, chunk fan-out             | <code>../08-rag/</code> — <code>Send</code> is map-reduce over chunks        |
| Traces, per-node latency and cost          | <code>../langsmith/</code> — LangGraph emits these for free                  |
| Chat models, tools, message types          | <code>../langchain/</code> — the layer below                                 |